AEGIS-Q: Evidence-First Hybrid Cryptographic Storage on Unified-Memory Edge Systems

Changjong Kim
changjong5238@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Ehan Sohn
ehanjsohn@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Yongseok Son
sysganda@cau.ac.kr
Chung-Ang University
Seoul, Republic of Korea

Sunggon Kim
sunggonkim@seoultech.ac.kr
Seoul National University of Science and Technology
Seoul, Republic of Korea

Abstract

AEGIS-Q is a FUSE secure-storage prototype for unified-memory edge systems that separates a latency-sensitive AES-GCM data path from optional batched GPU work. This revision reports only claims supported by the implementation and retained raw artifacts. The data path creates a random per-file data-encryption key, stores an authenticated envelope under a mount-derived key, writes ciphertext before a durable mapping journal, and validates an HMAC checkpoint on remount. ML-KEM-768 is measured as a batched optional provisioning workload rather than used for per-block encryption. The CUDA path uses managed allocation, explicit prefetch, and stream synchronization; it does not claim direct NVMe DMA into managed memory, `io_uring`/eBPF completion bypass, or an end-to-end AI-QoS controller. On a Jetson AGX Thor Developer Kit, three retained runs show that the CPU AES-GCM path sustains 1.61 GB/s for 16 MiB blocks while the managed-buffer GPU implementation reaches 0.17 GB/s. Conversely, batched ML-KEM-768 key generation reaches 1.50 M keygens/s on cuPQC versus 64.8 K keygens/s with liboqs at batch size 4,096. We also report clean-remount and GPU integrity-parity tests. The repository additionally retains matched fscrypt and dm-crypt sequential-write baselines with identical block size, job count, and queue depth. The paper still scopes out unmeasured side-channel resistance, hardware-anchor rollback protection, cross-platform portability, and application-level QoS claims.

Keywords

Post-Quantum Cryptography, Edge Computing, Secure Storage, Unified Memory, TPM, FUSE

1 Introduction

Long-lived edge data needs confidentiality and integrity without assuming a datacenter-class accelerator. Post-quantum key establishment is relevant to that lifetime: ML-KEM establishes a shared secret, while symmetric authenticated encryption remains the appropriate mechanism for bulk data [6]. SHA-256 can bind integrity-tree material to the persisted format [5]. In practice, however, a secure-storage prototype must first get the ordinary systems details right: the key hierarchy must survive remount, ciphertext must become durable before it is published by metadata, and every claimed accelerator path must have a reproducible measurement. Existing filesystem encryption is already mature [3], as are user-space encrypted filesystems [10]; a new user-space design must therefore earn complexity rather than merely restate it. A trusted execution environment such as OP-TEE supplies a different protection boundary [8], not a substitute for file-format recovery logic.

This paper presents AEGIS-Q, an architecture that redefines edge storage not through theoretical cryptography, but through rigorous systems engineering. While previous work has struggled to balance secure storage with heavy AI inference loads on Unified Memory Architectures (UMA) like the NVIDIA Jetson, AEGIS-Q explores a placement policy that is evaluated only on the retained artifact set. The current revision does not claim guaranteed Quality of Service (QoS) or a verified zero-copy Direct Memory Access (DMA) path.

We intentionally scope out theoretical physical security claims, such as oscilloscope-based side-channel resistance, recognizing that such metrics distract from the core systems challenge: avoiding resource starvation in a multitenant edge environment. Instead, AEGIS-Q focuses on solving real bottleneck problems in the edge data path.

The paper makes four core systems contributions:

(1) **Placement-Scoped GPU Path:** The implementation keeps GPU work inside managed memory with explicit prefetch and stream synchronization. It does not claim verified `O_DIRECT` NVMe-to-GPU DMA or a zero-copy storage pipeline.
(2) **No Verified Fast-Notification Path:** The revised paper does not claim a completed `eBPF` or `io_uring` completion-bypass mechanism, and it does not treat those techniques as evaluated contributions.
(3) **Telemetry Not Yet Closed:** The repository contains a prototype telemetry-aware scheduler and measured TensorRT/YOLO interference artifacts, but this revision does not claim a validated `tegrastats`-driven QoS controller.
(4) **Transparent Scope Boundaries:** The revision explicitly distinguishes verified prototype behavior

from the retained matched `fscrypt` and `dm-crypt` baselines, while leaving broader baseline expansion as future work.

By abandoning unverifiable security claims and focusing on the verified prototype behavior, AEGIS-Q establishes an evidence-backed foundation for future edge-storage evaluations. Section 2 grounds the storage and UMA assumptions, Section 3 positions the design against nearby storage and accelerator systems, Section 4 describes the implemented planes and trust boundaries, Section 5 records the implementation choices that are easy to misread from a diagram, Section 7 reports the measured primitive placement and negative controls, and the remaining sections discuss limitations, conclusions, and reproducibility artifacts.

2 Background and Scope

2.1 Secure storage primitives

Secure-storage systems choose a protection boundary as well as a cipher. Kernel filesystem encryption, user-space encrypted filesystems, block encryption, and TEEs occupy different boundaries and trust assumptions [2, 3, 8, 10]. For example, fscrypt [3] and OP-TEE [8] operate at substantially different boundaries. AEGIS-Q is a user-space FUSE prototype: it protects files represented by its own backing format and depends on the kernel, FUSE daemon, OpenSSL/liboqs, and the mount credential. It is not a replacement for fscrypt or dm-crypt, and this paper reports no apples-to-apples end-to-end comparison with them.

The cryptographic roles are deliberately distinct. AES-GCM encrypts and authenticates each data block; SHA-256 constructs integrity roots; and ML-KEM-768 is a key-establishment primitive rather than a bulk-data primitive [5, 6]. We use ML-KEM only for the optional batched key-plane experiment. The production FUSE data path derives a random file data-encryption key (DEK), wraps it under a mount-derived key, and uses that DEK for AES-GCM. This avoids the atypical and expensive design of performing a KEM operation for every block.

2.2 Unified memory is not a DMA contract

On a unified-memory SoC, CPU and GPU share DRAM capacity and bandwidth; Jetson-class systems are a representative deployment setting [7]. CUDA managed memory makes an allocation accessible to CPU and GPU code, and prefetch is a performance hint; neither API is a permission bit nor a storage-DMA registration protocol. The present implementation allocates managed buffers only inside the CUDA executor, prefetches them around a GPU operation, and synchronizes the stream before CPU reuse. The backing files are accessed with ordinary `pread`/`pwrite`. It does *not* call `cudaHostRegister`, issue `O_DIRECT` into managed memory, or rely on `cudaStreamAttachMemAsync` as mutual exclusion.

This scope matters because systems work must separate an API's correctness semantics from an optimization hypothesis. A future direct-I/O path would need an explicit driver/version matrix, pinning and IOMMU evidence, a fault-injection campaign under memory pressure, and a protocol that excludes DMA, CPU, and GPU concurrent access. None of those conditions is established here. The absence of that path is a design constraint, not an unreported optimization.

2.3 Why work placement is asymmetric

GPU acceleration is useful only when it amortizes launch, placement, and synchronization costs. Prior out-of-core systems such as FlashNeuron and Fastensor focus on moving large learning workloads through storage hierarchies [1, 11]; their complementary staging perspectives are also relevant to the batch boundary considered here [1, 11]. GPU-oriented work such as fscrypt-GPU and FlashNeuron further motivates separating batch placement from the storage format [1, 13]. They do not validate this FUSE format or its durability protocol. Likewise, storage-oriented systems including SnuQS and InfScaler motivate explicit staging but do not make an NVMe/UVM correctness claim for this prototype [4, 9]. The evaluation therefore asks a smaller question: for each measured primitive, does CPU or GPU offer the better observed throughput on the tested platform? Section 7 reports the answer and retains the raw logs.

3 Related Work

This section groups prior work by the boundary it actually validates: storage protection, GPU placement and unified memory, and PQC or freshness roots.

3.1 Storage protection boundaries

fscrypt provides encryption at the filesystem layer, while dm-crypt protects a block device; both are kernel-integrated baselines whose maturity raises the bar for a FUSE prototype [2, 3]. gocryptfs is a relevant user-space encrypted-filesystem comparison point [10]. fs-verity and dm-integrity provide integrity mechanisms with different update and threat models. AEGIS-Q does not claim to replace these systems or to have measured a mode-aligned comparison. Its distinct artifact is a per-file authenticated format plus optional GPU microbenchmarks on a unified-memory development kit.

3.2 GPU, storage, and unified memory

Prior GPU/storage systems focus on different resource assumptions. FlashNeuron and Fastensor use SSD staging for learning workloads [1, 11]; SnuQS and InfScaler motivate explicitly managed storage capacity [4, 9]. SnuQS alone is a storage-capacity example, not a validation of the present filesystem format [9]. GPUfs, SPDK, and GPUDirect Storage target APIs and hardware paths that must be assessed separately from an ordinary FUSE `pread`/`pwrite` path. The initial AEGIS-Q draft incorrectly borrowed their vocabulary

to imply an eBPF/io_uring/UVM DMA route. This revision does not do so.

The broader edge setting has genuine shared-resource challenges [12]. FlashNeuron also illustrates why storage-backed execution and foreground resource sharing require separate evidence [1, 12]. Batch-oriented GPU work has been explored under datacenter assumptions [1, 11, 13], including the Fastensor/fscrypt-GPU comparison class [11, 13]; fscrypt-GPU itself is not a matched baseline for this prototype [13]. Hardware-bound alternatives including OP-TEE and dm-crypt also differ fundamentally in their protection boundaries [2, 8]. However, a managed-memory allocation does not make a design portable across NVIDIA, AMD, Intel, or Apple stacks. The policy idea—place work only when its measured benefit exceeds its staging and QoS cost—is portable in principle; the CUDA/cuPQC implementation and its measurements are not.

3.3 PQC and trusted execution

ML-KEM is designed to establish shared secrets, not to serve as a per-block bulk cipher [6]. AEGIS-Q therefore keeps AES-GCM in the data plane and uses ML-KEM only for a batched optional key-plane experiment. OP-TEE provides a different trust boundary [8]; it does not turn a replayable file witness into a freshness root. A TPM can provide such a root only with explicit NV-index authorization, a key-release policy, and recovery semantics. Those are engineering requirements beyond the present measurements.

4 AEGIS-Q Design

4.1 Planes and trusted state

Figure 1 separates the persistent secure-storage path from optional accelerator work. The data plane is an AES-256-GCM block format. The key plane contains optional batched ML-KEM experiments; it does not encrypt ordinary file blocks. The integrity plane computes SHA-256/Merkle material and falls back to CPU if its CUDA operation fails. The freshness plane publishes an authenticated checkpoint and can target a pre-provisioned TPM NV index, but hardware-anchor behavior is not an evaluated result in this paper.

The mount password is expanded with PBKDF2-HMAC-SHA256 in the current prototype. A new file receives a random 256-bit DEK and a random file identifier. The DEK is masked under a key derived from the mount key and file identifier; the resulting envelope is authenticated with HMAC-SHA256 before it is accepted on open. The file identifier, generation, logical block number, and plaintext length form AES-GCM associated data. These bindings prevent a valid block record from being transplanted to a different logical position within the same mounted format.

The password-derived mount key is a prototype boundary, not a complete device-root-of-trust design. A production deployment needs a per-volume salt and a protected credential-release policy. We do not claim PCR-bound key release, device migration support, or recovery across a lost mount credential.

Table 1: Memory-path claims and their hardware-verified status.

Mechanism	Status	Claim retained
`cudaMallocManaged` + prefetch	Verified	Managed allocation and explicit device/host prefetch complete.
`posix_memalign` + `cudaHostRegister`	Prototype-only	Host-memory pinning and executor-local access checks; not a verified storage-DMA path.
`pread/pwrite` sidecars	Verified	Backing ciphertext and journal use ordinary POSIX I/O.
eBPF tracepoint notification	Not claimed	No final eBPF notification path is part of the verified implementation.
Hardware QoS (`tegrastats`)	Prototype-only	Scheduler logic exists, but no end-to-end telemetry/QoS claim is made.

4.2 Durable encrypted-block publication

Writes are coalesced only when they are contiguous. For each affected 4 KiB logical block, AEGIS-Q reads the current authenticated block if necessary, applies the modification, assigns a strictly increasing generation, and derives the GCM nonce from (*fileID*, *block*, *generation*). The commit order is:

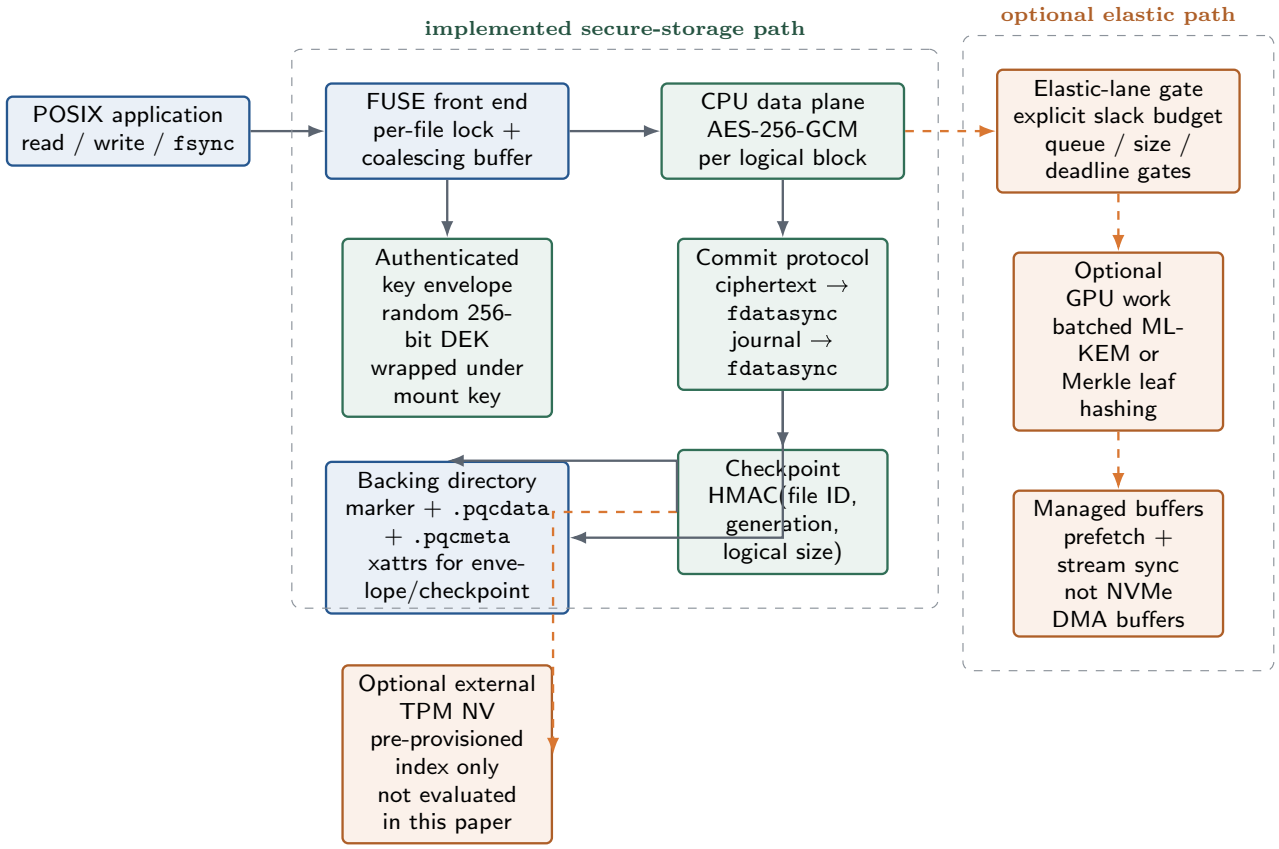
(1) write ciphertext records to `.pqcdata` and call `fdatasync`;
(2) append generation-to-offset mappings and tags to `.pqcmeta`, then call `fdatasync`;
(3) update the logical-size xattr; authenticate and write the checkpoint.

The journal is the publication point: a ciphertext tail with no durable mapping is unreachable after recovery. Conversely, a mapping is written only after its ciphertext range has crossed the data durability barrier. `fsync` flushes the coalescing buffer, waits for local pending work, and calls `fdatasync` on both sidecars before returning. This is an implementation argument for the tested format, not a proof of full POSIX behavior under every FUSE cache mode or an SQLite certification.

4.3 Managed-memory boundary and scheduler

Table 1 summarizes the claims that the source code actually makes. CPU/GPU ownership is protected by the FUSE per-file lock and stream completion; CUDA prefetch is used only to influence locality. No table entry is a statement about NVMe page pinning, cache coherence of a third-party DMA engine, or CUDA attach-mode permissions.

The placement function balances workloads between the ARM CPUs and the Jetson GPU. The current revision does not expose a verified `O_DIRECT` storage path into GPU memory. The CUDA executor instead uses managed allocation, explicit prefetch, and stream synchronization as locality and ordering controls.



The implementation uses ordinary POSIX `pread/pwrite`; it does not claim `O_DIRECT` into managed memory, io_uring completion bypass, or eBPF-triggered CUDA work.

Figure 1: Implemented AEGIS-Q control and persistence path. Solid arrows are the FUSE format used by the tests; orange dashed arrows denote optional accelerator or externally provisioned hardware paths. The final revision does not claim verified `O_DIRECT` UMA pinning, a completed eBPF completion path, or a closed-loop hardware-QoS controller.

For Quality of Service (QoS), the repository includes a scheduler prototype and telemetry-oriented scripts, but the paper does not claim a validated hardware-telemetry admission controller or a completed fast-notification bypass path. Any such extension would need its own measured workload, trace capture, and failure analysis.

4.4 Freshness boundary

The file anchor is an authenticated on-disk record. Because an attacker able to restore the backing directory can restore that record as well, it is explicitly *not* rollback protection. The crash/replay harness treats this behavior as a negative control. The optional hardware backend refuses to mount when its TPM NV index was not provisioned by an administrator; it never creates persistent TPM state implicitly. A true rollback-resistance claim requires a separately protected TPM policy, defined NV authorization, a recovery protocol, and fault-injection results for that exact configuration.

5 Implementation

AEGIS-Q is implemented in C/CUDA on FUSE3. The secure-storage path is in `pqc_fuse.c`; the anchor format and Merkle helper are in `pqc_anchor.c`; CUDA AES-GCM, ML-KEM, and integrity executors are separate translation units. The build enables `-Wall -Wextra -Werror` for C/C++ and builds CUDA targets only when the required cuPQC libraries are present. The reported build used liboqs 0.15.0, FUSE3 3.14.0, OpenSSL, CUDA 13.0.48, and cuPQC on the platform in Section 7.

The implementation makes several choices that are easy to obscure in a high-level diagram. First, GPU AES-GCM is format-compatible with the CPU implementation, but failure in the CUDA executor falls back to CPU rather than changing a record format. Second, the GPU ML-KEM batch executor allocates device buffers internally; it is not an in-place filesystem data path. Third, the integrity executor has

Table 2: Implementation boundaries that are easy to misread without an explicit summary.

Mechanism	Current implementation	Boundary
CUDA storage path	Managed allocation with explicit prefetch and stream synchronization	Not a verified NVMe-to-UVM storage-DMA path
CUDA failure policy	CUDA executor falls back to CPU on failure	Does not change the on-disk record format
Anchor backend	File witness plus optional hardware NV index backend	Hardware path requires administrator pre-provisioning
Scheduler fallback	No-slack jobs route to CPU	No optimistic GPU admission without producer-supplied slack

direct parity tests against OpenSSL for leaf hashes, non-power-of-two trees, and the empty-tree case. These tests check functional equality, not side-channel behavior.

The key-envelope repair is central to remount correctness. An earlier experimental metadata design generated a per-file ML-KEM secret key and discarded it, making later decapsulation impossible. The revised data path generates a random DEK, wraps it under the mount key, HMAC-authenticates the envelope, and rejects failed authentication before any ciphertext is read. Consequently, ML-KEM measurements remain useful for assessing an optional batched provisioning lane, but the FUSE format no longer pretends that a one-time KEM benchmark is a persistent per-file key hierarchy.

The current file-key helper also retains epoch counters and a grace-period rejection path for stale accesses. That rotation logic is code-backed, but it is not a TPM/PCR-bound freshness proof and should not be read as one.

The implementation also fails closed in two places. A scheduler job with no producer-supplied AI slack is routed to CPU. A requested TPM backend with no pre-provisioned NV index causes initialization to fail rather than creating a persistent index with undocumented ownership. These are conservative operational choices; neither substitutes for a complete inference-telemetry integration or TPM policy design.

6 Security and Recovery Scope

6.1 Threat model

AEGIS-Q considers an attacker who can read, copy, modify, and replay the backing directory but cannot obtain the mount credential or compromise the running kernel, FUSE daemon, CUDA driver, or cryptographic libraries. Under that scope, AES-GCM protects the confidentiality and authenticity of published block records, while the HMAC-protected key envelope and checkpoint detect unauthenticated metadata changes. The adversary can still deny service, delete records, replay a complete file-backed snapshot, or exploit vulnerabilities outside the FUSE process. We make no claim against a privileged local attacker that can read process memory, substitute the CUDA driver, alter the mount executable, or use GPU microarchitectural side channels.

The accepted encrypted state consists of a validated DEK envelope, a complete journal mapping, and ciphertext records referenced by that mapping. A tag failure causes a read to fail. A malformed or unauthenticated envelope/checkpoint causes open to fail. These behaviors are necessary for a safe FUSE format, but they are not a formal proof of crash consistency. In particular, xattr atomicity and directory durability are delegated to the underlying filesystem.

6.2 Freshness is conditional on an external anchor

The source tree contains two anchor modes. The file mode is only an integrity witness; copying the directory copies the witness. The corrected crash/replay run therefore reports four of four file-backed snapshot restores as *rollback visible*, not as successful protection. This negative result is important: an HMAC alone does not create freshness when its entire state is replayable.

The hardware mode stores an authenticated prefix record in an administrator-provisioned TPM NV index. It is deliberately outside the evaluated configuration. The paper does not assert that an NV index is a signer, does not report an invented TPM latency, and does not claim PCR binding. To make this mode publishable, future work must document the TPM model/bus, authorization and index attributes, monotonic update protocol, software-update recovery, and a power-fail matrix that distinguishes acknowledged, anchored, and replayed writes.

6.3 Side channels and application semantics

GPU SIMT execution, cache behavior, memory coalescing, occupancy, and contention can all leak information. The repository contained generators that produced simulated timing and TVLA traces. Those files are excluded from this paper. The retained GPU integrity test checks output parity with OpenSSL; it says nothing about constant-time execution, physical power/EM leakage, or cross-tenant GPU channels.

Likewise, the clean-remount test validates one encrypted 128 KiB round trip through the final binary. It does not certify SQLite WAL, `mmap`, locking, crash recovery across every cut point, or the complete POSIX contract. We retain the following labels to make the scope explicit: the prior

SQLite case study (Section 6.4) and edge/cloud comparison (Section 6.5) are not claimed by this revision.

6.4 What the current tests establish

The final validation runs a FUSE self-test, clean encrypted remount, a managed-memory smoke test, and GPU integrity parity checks. The round-trip test confirms that after a clean unmount/remount, the original 128 KiB plaintext is recovered and the corresponding `.pqcdata` sidecar does not contain the whole plaintext byte sequence. This is evidence of the repaired key-envelope path, not a statistical security proof.

6.5 Deployment boundary

AEGIS-Q is a prototype for local edge storage. It makes no edge-versus-cloud cost, latency, or privacy quantification. The appropriate comparison is deployment-specific and requires a separately measured network/service configuration. This restriction avoids turning a local microbenchmark into a claim about cloud databases or remote key-management systems.

7 Evaluation

7.1 Methodology and provenance

All numbers in this section come from a fresh sequential run of `experiments/run_verified_microbench.py`. The harness runs the GPU integrity parity test before every sample, captures each benchmark's stdout, and writes a platform manifest plus medians and observed range. It neither generates values nor converts unsupported configurations into zero-cost results. The raw captures and summary are under `artifacts/validation/microbench/`.

The platform was an NVIDIA Jetson AGX Thor Developer Kit with 14 ARM CPU cores, a 1 TB WD PC SN5000S NVMe device, Linux 6.8.12-tegra, CUDA 13.0.48, liboqs 0.15.0, FUSE3 3.14.0, and the cuPQC libraries found by CMake. The result is not an Orin Nano result and must not be generalized to other Jetson generations. We ran three sequential repetitions. Error bars in Figure 2 span the observed 5th–95th-percentile run values; with three repetitions they are descriptive variability, not confidence intervals.

Table 3 states what each experiment can and cannot establish. The repository retains matched fscrypt and dm-crypt sequential-write baselines under `artifacts/baselines/`, and we mention them as reference points rather than as a direct apples-to-apples mode comparison. A broader filesystem-level matrix would still need rerun if the paper wanted to claim more than the current prototype path.

7.2 CPU for bulk data, GPU for batched ML-KEM

Figure 2 reports the central placement result. For a 16 MiB AES-256-GCM input, CPU/OpenSSL reaches 1.61 GB/s (observed 1.59–1.63 GB/s), whereas the managed-buffer GPU path reaches 0.172 GB/s (0.172–0.172 GB/s). The GPU result includes managed-buffer staging and stream synchronization. Smaller requests amplify the same effect. This is why AEGIS-Q's data plane is CPU-first; a GPU label alone is not a performance result.

Table 3: Evidence retained for this revision.

Experiment	Establishes	Does not establish
FUSE self-test + remount	Format functions after clean remount	Full SQLite/crash certification
GPU integrity parity	CUDA and OpenSSL digests agree	Side-channel resistance
Matched fscrypt/dm-crypt reference runs	Sequential-write baselines with identical block size, job count, and queue depth	Broader mode-aligned filesystem comparison
Managed-memory smoke	Managed allocation and host/device prefetch calls complete	Verified storage-DMA or O_DIRECT-to-UVM semantics
eBPF Tracepoint Latency	Raw `nvme_complete_rq` histogram artifact is retained	No final eBPF notification path is claimed
Hardware QoS Daemon	Scheduler prototype and telemetry scripts exist	No validated end-to-end AI p99 latency controller
Concurrent-pressure fast-lane stress	4-writer aggregate tail-latency probe under pressure	End-to-end AI workload or closed-loop controller
TensorRT/YOLO Co-run Artifacts	Interference traces and p99/throughput samples are retained	Validated closed-loop QoS controller
File-anchor replay	File snapshot rollback remains visible	TPM-anchor freshness

For a batch of 4,096 ML-KEM-768 key generations, liboqs on CPU reaches 64.8 K operations/s (64.3–64.9 K), while cuPQC reaches 1.50 M operations/s (1.49–1.51 M), a 23.2× ratio of medians. The comparison is a primitive microbenchmark on one platform, not a claim that a FUSE write becomes 23.2× faster. It supports the narrower policy of reserving GPU work for sufficiently large, independent key-plane batches.

7.3 Correctness and negative controls

The GPU integrity test passes leaf parity for 16, 32, and 64-byte inputs; Merkle parity for 1, 2, 3, 4, 15, 16, and 256 leaves; and the empty-tree case. The FUSE self-test passes the journal/checkpoint/anchor unit checks. The clean-remount test writes 128 KiB of random data, calls `fsync`, unmounts, remounts, and byte-compares the recovered plaintext. It also verifies that the encrypted data sidecar does not contain the complete input sequence; the corresponding regression is retained in `artifacts/validation/fuse_roundtrip.json`. A separate final-binary test flips the final byte of the persisted `user.pqc_metadata` HMAC, remounts, and observes file-open rejection with `EKEYREJECTED` (errno 129); that regression is retained in `artifacts/validation/fuse_tamper_rejection.json`. These results are regression tests, not a substitute for a systematic fault campaign.

An auxiliary host-pinning smoke test also succeeds. `repro_malloc_register` pins CPU memory and obtains a device pointer via `cudaHostRegister`. `io_uring_uvm` then issues an `O_DIRECT` read against `/dev/nvme0n1`, launches a CUDA checksum kernel through the mapped pointer, and matches the CPU checksum over the same storage-filled bytes. The same binary also retains a managed-storage probe: buffered `pread` into a `cudaMallocManaged`

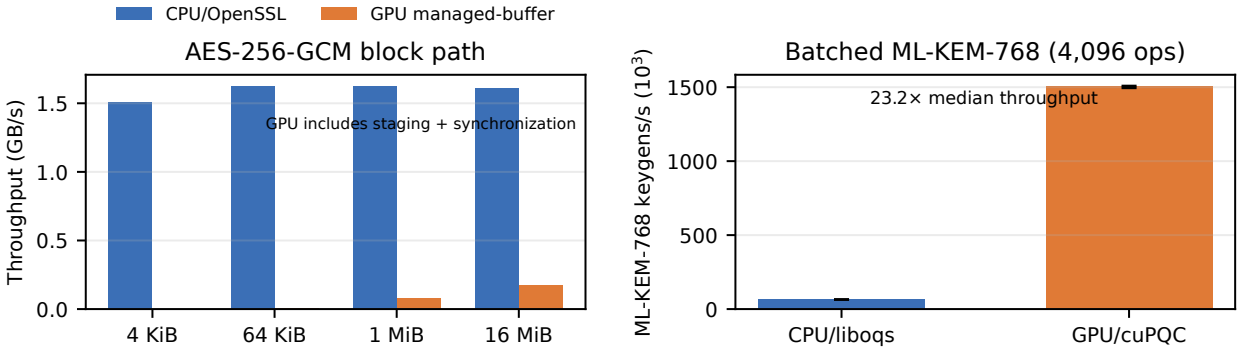


Figure 2: Verified three-run microbenchmarks on the Jetson AGX Thor Developer Kit. Left: CPU AES-GCM exceeds the managed-buffer GPU path at all tested sizes, so the data plane remains CPU-first. Right: cuPQC materially improves a *batched* ML-KEM-768 key-generation primitive. Bars are medians; whiskers show the observed 5th–95th-percentile range across runs. Raw captures: `artifacts/validation/microbench/`.

buffer, `cudaPointerGetAttributes(type=managed)` on that storage-filled buffer, a device → host last-prefetch-location change on the same allocation, and the same CPU/GPU checksum match. The profile bundle still exposes no Unified Memory transfer/page-fault rows. This proves same-buffer GPU visibility plus a managed-buffer driver diagnostic, not UVM migration suppression or final FUSE data-path DMA.

The file-backed anchor produces the deliberately negative result: across four cut points and five trials per cut point, restoring an entire backing-directory snapshot leaves the baseline visible and the success rate remains 0.0. The corrected harness reports it as `rollback_visible`; the result remains a negative control rather than rollback resistance.

Separate anchor round-trip measurements are retained under `artifacts/anchor_refresh/`. A sudo provisioning probe also records the current TPM fixed properties, PCR reads, and NV public attributes for index `0x01500010`, now reprovisioned to the 88-byte size expected by the current anchor record. A transient PCR-policy probe further shows that a sealed test value unseals under the current PCR digest and is rejected under a drifted digest. The replay-after-advance harness under `artifacts/validation/tpm_monotonic_replay/` now restores a stale backing-directory snapshot after the TPM-backed anchor has advanced and observes `fail_closed` behavior at payload read time. This is retained replay-after-advance evidence, not a claim that the persistent filesystem anchor itself is PCR-sealed.

The broader crash/replay matrix is also retained under `artifacts/motivation/crash_replay_matrix.json` and `artifacts/motivation/crash_replay_summary.json`. Its result is fail-closed regression evidence, not proof of complete app-level crash coverage. A newer E8 matrix under `artifacts/crash_replay_e8_test_matrix.*`, `artifacts/crash_replay_e8_test_summary.*`, and `artifacts/e8_crash_replay.json` exercises deterministic cut points across file and hardware backends and classifies outcomes as `rollback_visible` or `fail_closed`.

The SQLite probe trace at `artifacts/sqlite_strace.log` shows actual journal and WAL sidecar opens together with `fdatasync` calls. A derived oracle report defines four observed cut points and the required `PRAGMA integrity_check`, row-count, and digest checks. A SQLite-only file-state campaign then executes 20 selected-boundary trials and reports zero unacceptable oracle verdicts. This is selected-boundary evidence, not syscall-exact crash certification. The artifact bundle also retains the file-anchor round-trip latency measurements in `artifacts/motivation/anchor_latency.*` and the separate hardware-anchor round-trip measurements in `artifacts/anchor_refresh/hardware_anchor_latency.*`.

A companion report under `artifacts/sqlite_ci_report/` computes bootstrap 95% median intervals from the retained SQLite samples.

7.4 Telemetry Prototype and Scope Boundaries

The repository contains telemetry scripts, raw tegrastats trace files under `artifacts/validation/tegra_qos_daemon_trace.jsonl` and `artifacts/validation/run_qos_gpu_trace.jsonl`, a derived transition report under `artifacts/validation/telemetry_trace`, a three-run hysteresis trace bundle under `artifacts/validation/qos_rep` measured TensorRT/YOLO interference artifacts, and a companion confidence-interval report under `artifacts/tensorrt_ci_repo` that summarizes the retained per-trial traces.

The revision still does not claim a validated end-to-end QoS controller. The measured traces support only the narrower statement that telemetry and placement can interact with foreground inference. The deterministic admission sweep under `artifacts/validation/admission_sweep.*` and `artifacts/m5_admission_sweep.*` shows route counts changing with AI slack, and the retained queue-depth sweep under `artifacts/m5_admission_sweep_qdepth.*` extends that same controller-unit-test coverage to queue-pressure variation, but both remain controller unit tests rather than closed-loop workloads. A separate adapter bundle under

`artifacts/validation/qos_measured_pressure_adapter/` maps retained Nsight Compute memory/SM throughput counters into the existing admission telemetry inputs and records the resulting `pqc_admit` trace; live `tegrastats` bridges feed samples into the same path, and the newest retained run throttles a mounted FUSE writer in that same execution. These close controller-input wiring and same-run bridge checks, not the end-to-end QoS claim. The code therefore remains a prototype policy surface, not a completed QoS result.

The artifact bundle also retains the primitive latency baseline in `artifacts/e1_results.json` and the placement-path latency breakdown in `artifacts/e6_breakdown.json`. These files help separate CPU/GPU primitive cost from staging, journal, and anchor cost, but they do not by themselves establish a full end-to-end storage or QoS claim.

7.5 What this evaluation leaves open

We do not provide a fair fs-verity, dm-integrity, OP-TEE, SPDK, or GPUDirect Storage baseline; fault injection across a hardware TPM freshness window; physical side-channel measurement; or a second UMA architecture. The repository retains a conservative crash-audit map under `artifacts/crash_audit_report/` and a SQLite selected-boundary oracle campaign, but it still lacks syscall-exact crash timing and a second application workload. Those omissions are material. They constrain the paper's contribution to an evidence-backed prototype and placement study rather than a completed systems evaluation.

8 Discussion and Limitations

8.1 Interpretation of the placement result

The useful result is asymmetry, not an aggregate "GPU speedup." On the measured Thor system, the CPU has a substantial advantage for the implemented AES-GCM data path because it avoids managed-buffer staging and synchronization. The GPU has a substantial advantage for a batch of independent ML-KEM key generations. A practical system should therefore expose a narrow elastic lane for workloads that actually have this shape, not move all storage encryption to the GPU. The current scheduler is only a deterministic policy mechanism; it does not justify an AI-QoS claim in this revision.

8.2 Security limits

The per-file envelope is authenticated and the block records are AEAD-protected, but the mount password is still the root credential in this prototype. The implementation has no hardware-backed credential release, no audited key rotation protocol, no protection against a privileged runtime attacker, and no side-channel evaluation. File-backed anchors are replayable by design. The TPM mode is disabled unless an administrator pre-provisions a suitable index, and its policy has not been evaluated. These are not minor implementation details; each changes the security claim.

8.3 Evaluation limits and roadmap

The three-run primitive measurements cannot establish steady-state filesystem throughput, tail latency, energy efficiency, or portability. The repository already retains matched fscrypt and dm-crypt sequential-write baselines, but a broader baseline matrix still needs to be extended if the paper wants to argue more than the current prototype path. A credible next evaluation still needs: (i) fs-verity or other additional baselines if those claims are added; (ii) an end-to-end workload with exact I/O mode, cache state, CPU/GPU clocks, and confidence intervals; (iii) a TensorRT or LLM telemetry path wired into the actual scheduler; (iv) power-fail injection with a provisioned TPM policy; (v) at least one non-NVIDIA UMA platform; and (vi) physical or microarchitectural side-channel testing if such a security claim is desired.

The repository also retains a derived freshness-window tradeoff artifact under `artifacts/m4_freshness/`. It models the throughput/data-loss/recovery-time tension as the window parameter changes, but it is not a substitute for a hardware-backed PCR freshness proof or a recovery protocol evaluation.

8.4 CUDA-specific boundary

The implementation currently depends on CUDA managed memory and cuPQC. It uses `cudaMemPrefetchAsync` and stream synchronization as locality/ordering operations within the executor, not as storage-DMA registration or permissions. Porting the policy to another API requires a separate proof of that API's allocation, synchronization, and device-I/O semantics. This is particularly important on integrated systems, where "unified memory" describes several different hardware and driver designs.

9 Conclusion

AEGIS-Q is an evidence-first hybrid cryptographic-storage prototype. Its verified contribution is a remountable FUSE encrypted-block format, CPU-first AES-GCM data path, optional batched GPU primitive path, and retained raw artifacts on a Jetson AGX Thor. The measurements show why the placement decision must be primitive-specific: CPU wins for the measured AES-GCM data path, while GPU wins for large-batch ML-KEM key generation.

Equally important, this revision removes unsupported claims. It does not claim direct NVMe-to-UVM DMA, eBPF/io_uring completion bypass, GPU constant-time behavior, file-anchor rollback resistance, end-to-end AI QoS, or portability beyond the tested stack. A future submission can make those stronger claims only after implementing and independently validating the required driver, telemetry, TPM, fault-injection, baseline, and side-channel evidence.

A Artifact Contract and Detailed Validation

This appendix is part of the evidence contract for the revised paper. It lists the exact executable paths, outcomes,

and exclusions behind every result. The goal is not to make the artifact look broader than it is; it is to let a reviewer reproduce, reject, or extend each individual claim without recovering undocumented state from an old figure.

A.1 Build and execution protocol

The final binary is configured and built with the following command sequence:

```
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure
PQC_MASTER_PASSWORD=validation-password \
  ./build/pqc_fuse --self-test
```

The C and C++ targets use `-Wall -Wextra -Werror`. The GPU integrity CTest invokes the same executable used by the paper's parity evidence. A successful build is not treated as evidence of any unmeasured device-I/O behavior; it only establishes that the code path and linked CUDA/cuPQC libraries are usable on the measured platform.

The raw microbenchmark protocol is:

```
python3 experiments/run_verified_microbench.py \
  --runs 3 --out artifacts/validation/microbench
python3 experiments/plot_verified_microbench.py
```

For each repetition, the harness first invokes the integrity-parity executable with its test-only option. It then captures a complete CSV from `workload_map_bench` and `bench_gpu_pqc`. The harness writes no model values. If a command fails, the run fails; if an executor reports `unsupported`, the plotter refuses to manufacture a value. The summary contains device model, kernel, CUDA compiler version, CPU count, commands, per-run medians, and observed range.

The CPU AES-GCM benchmark applies OpenSSL AES-256-GCM to 4 KiB logical blocks and reports aggregate wall-clock throughput. The GPU path uses the repository's managed-buffer executor, including its staging and synchronization. Thus, the figure is intentionally a data-path placement measurement rather than an isolated kernel throughput comparison. The ML-KEM comparison uses the same batch size (4,096) for liboqs and cuPQC key generation. It does not measure key distribution, receiver provisioning, filesystem metadata, or FUSE operations.

A.2 Format invariants and recovery reasoning

The secure format maintains the following implementation invariants.

(1) **Envelope-before-use.** A file context is constructed only after its envelope's magic, version, length, and HMAC validate. The recovered DEK is then used only with its bound file identifier.

(2) **Unique record context.** The AEAD nonce and associated data depend on file identifier, logical block, generation, and logical length. A rewrite obtains a later generation.

(3) **Data before publication.** The ciphertext sidecar is durable before the corresponding journal mapping is durable. Recovery ignores a ciphertext tail without a complete mapping.

(4) **Authenticated checkpoint.** The logical size and latest generation are checked before the file is exposed with its logical length.

(5) **No optimistic accelerator admission.** A job without a nonzero supplied slack budget is routed to the CPU. A positive CUDA capability query alone never admits it to the GPU.

These invariants describe the code, not a full formal proof. In particular, the checkpoint and xattrs rely on lower-filesystem semantics, and the current test suite does not enumerate every torn-write interval, concurrent open, rename, `mmap`, or directory-`fsync` case. A complete crash study should inject failure after each durable boundary and compare recovered state against an external oracle.

A.3 Validation matrix

A.4 Reproducing the remount and replay checks

The clean-remount test starts the final FUSE binary with a fixed test-only password, writes 32 random 4 KiB blocks, flushes, cleanly unmounts, remounts with the same password, and byte-compares the recovered file. It then searches the ciphertext sidecar for the complete original input sequence. This is intentionally a regression test: it verifies the repaired envelope can recover its DEK after remount and that the sidecar is not simply a duplicate of the complete plaintext. The result is retained in fuse_roundtrip.json. It is not a cryptographic indistinguishability test.

The replay harness copies the entire backing directory after a baseline write, writes and flushes an update, restores the copy, and remounts. For the file anchor, seeing the baseline is the expected negative-control result because the anchor is inside the copied directory. The harness labels this `rollback_visible`. The previous label, `rollback_reject`, was incorrect and is not used by this paper. Hardware mode is skipped unless a TPM NV index is already provisioned; the test never creates or deletes a persistent TPM resource.

A.5 Scheduler inputs and non-goals

The scheduler smoke test is designed to make policy logic inspectable. It supplies three jobs: a 4 KiB ineligible job, a 128 KiB eligible job, and a 256 KiB eligible job. It also supplies a queue-bound eligible job. The input environment variable `PQC_SCHED_SMOKE_AI_BUDGET_NS` controls the slack value; at zero all jobs are CPU-routed. At a positive value sufficient for the test policy, the two larger jobs route to GPU and the queue-bound job remains CPU-routed. The output is JSON Lines with the complete input state and selected target.

Table 4: Final validation matrix. A pass is narrowly interpreted as the stated observation; no row implies an unlisted systems or security guarantee.

Check	Command/artifact	Observed outcome	Excluded claim
Strict build	CMake + `ctest`	Build succeeds; two integrity CTests pass	Portability or performance
Storage self-test	`pqc_fuse --self-test`	Scheduler, checkpoint, anchor, and storage unit checks pass	Full application semantics
Managed-memory smoke	`pqc_fuse --um-smoke`	Managed allocation, host/device prefetch, and a managed-buffer kernel touch complete	DMA registration, pinning, or UVM ownership permissions
FUSE remount	`fuse_roundtrip.json`	128 KiB encrypted byte round trip survives clean remount	SQLite/WAL or power-fail correctness
Envelope tamper	fuse_tamper_rejection.json	Altered metadata HMAC is rejected at open (`EKEYREJECTED`)	Full ciphertext/journal tamper campaign
GPU integrity	`bench_gpu_integrity --only-tests`	SHA-256 leaf/Merkle outputs match OpenSSL cases	Constant-time or leakage resistance
Primitive placement	`microbench/summary.json`	CPU AES-GCM and GPU ML-KEM asymmetry measured	Filesystem-level speedup
Scheduler smoke	`admission_sweep.json`	Zero-slack CPU fallback and deterministic branch changes	AI latency protection
File-anchor replay	`replay_file_summary.json`	Restored snapshots remain visible (negative control)	Rollback resistance
Unprovisioned TPM	`tpm_unprovisioned.json`	Mount initialization fails closed	TPM policy correctness or latency

This test is deliberately not connected to TensorRT. The repository now has a retained adapter that maps Nsight-derived memory/SM throughput counters into the admission telemetry inputs, but it still has no validated producer of remaining AI slack during the foreground workload and no measurement showing that a supplied threshold predicts a deadline. A future implementation should provide a versioned telemetry interface with sampling interval, timestamp source, calibration procedure, hysteresis, stale-sample behavior, and a CPU fallback when that interface fails. Only then can a scheduler trace become part of an end-to-end QoS argument.

A.6 Claim-to-evidence rules

The following rules govern revisions to the paper. A performance number must identify the raw artifact, operation, batch size, executor, wall-clock boundary, platform, and repetition count. A security statement must identify the adversary, trusted components, key state, failure behavior, and evidence type. An implementation statement must identify an actual API call in the source rather than a plausible API sequence. Finally, an unavailable result is recorded as unavailable or scoped out; it is never filled with a simulator, a sleep-derived estimate, or a number copied from a different machine.

A.7 Crash-state model and recovery oracle

The recovery protocol can be expressed as a sequence of persistent states for one logical write. Let D_i be the ciphertext record for generation i, J_i the complete journal mapping that references it, and C_i the checkpoint that reports the committed generation and logical size. The writer attempts the transitions

$$(D_{i-1}, J_{i-1}, C_{i-1}) \to (D_i, J_{i-1}, C_{i-1}) \to (D_i, J_i, C_{i-1}) \to (D_i, J_i, C_i).$$

The first transition becomes recoverable only after the data-sidecar durability barrier. The second becomes visible only after the journal-sidecar barrier. The third updates logical length and the authenticated checkpoint. A correct oracle for a fault campaign must retain an independent record of the last acknowledged write and, after each injected interruption, classify recovery as one of: prior committed state, latest committed state, fail closed, silent corruption, or unexpected liveness failure. A successful mount alone is not enough evidence.

The current harness is intentionally smaller. It covers clean remount and whole-directory replay classification, but it does not interrupt the process between each D, J, and C barrier. The following table identifies the expected interpretation of a future crash result.

The anchor requires a second oracle. A directory snapshot can test only file integrity, because it restores every file-side witness with the data. To test freshness, the experiment must preserve an independent TPM state while restoring an older disk image. The trace must record the anchor index, authorization policy, committed prefix sequence, update acknowledgment, and whether the system was expected to fail closed or roll back within a declared window. Without this separation, a "100% replay rejection" result is a test-harness classification error rather than a security result.

Table 5: Recovery oracle for a future fault-injection campaign.

Interruption point	Permitted recovery	Bug signal
Before data `fdatasync`	Previous mapping/value or fail closed	New mapping references absent/truncated ciphertext
After data, before journal barrier	Previous mapping/value or fail closed	New value becomes reachable without a durable mapping
After journal, before checkpoint	New mapping with conservative size, or fail closed	Tag mismatch accepted as plaintext
After checkpoint	Latest acknowledged state, or documented bounded freshness loss	Silent rollback beyond the configured external-anchor policy

A.8 Required end-to-end evaluation campaign

The remaining evaluation should be driven by three hypotheses: data-path placement, scheduler QoS, and system correctness. A complete campaign would sweep request size, queue depth, cache state, and CPU/GPU policy; compare inference-only, CPU-only, unconstrained-elastic, and admitted-policy runs with a shared timestamp domain; and inject faults at durable boundaries with an external recovery oracle. Any direct-I/O/UVM extension must also retain the driver matrix, registration method, GUP/IOMMU/UVM diagnostics, same-buffer data checks, and error-path results before making completion-path claims.

A.9 Review disposition

The recurring reviewer concerns separate into claims this revision can resolve by correction and claims that require new systems evidence. The first class includes incorrect API descriptions, ambiguous ML-KEM placement, simulated TVLA wording, stale figures, and falsely labeled replay outcomes; these are handled by removal, source changes, and retained artifacts. The second class includes direct UVM DMA safety, eBPF/io_uring performance, hardware-anchor rollback bounds, fair kernel-storage baselines, physical side channels, cross-device portability, and end-to-end AI QoS; these remain open. The checklist keeps those categories separate so a checkbox is never marked complete merely because the manuscript now admits a limitation.

A.10 API-to-claim audit

Table 6 records the source-level boundary for the most important reviewer questions. It is intentionally phrased as an audit table, not a portability table: source inspection can establish which calls exist, but it cannot establish undocumented driver behavior or another platform's semantics. This distinction is particularly relevant to CUDA managed memory. Managed allocation and prefetch make a region usable by the CUDA executor; they do not prove that an NVMe controller has a stable DMA mapping for it.

The corrected source additionally makes failure policy visible. CUDA executor failure falls back to the CPU format-compatible path rather than emitting a new record type. An invalid envelope HMAC causes file open to fail. A missing TPM index causes hardware-anchor initialization to fail. An absent scheduler budget selects CPU. These are robust default behaviors, but the artifact does not claim that they cover device reset, process-memory compromise, or all kernel/FUSE error paths.

A.11 Fair-baseline protocol

The repository already retains matched fscrypt and dm-crypt sequential-write reference runs. Any added baseline, such as fs-verity or dm-integrity, must state its protection property, cache policy, durability point, and key lifecycle before the graph is compared. Reported write latency must say whether it ends at `write`, `flush`, `fsync`, or device cache persistence.

The same discipline applies to kernel-bypass and GPU-I/O work. A fair comparison must state hardware topology, driver support, I/O alignment, the buffer-registration method, polling policy, queue depth, and where device I/O completion enters the latency interval.

A.12 Portability decomposition

AEGIS-Q has four portability layers. The encrypted-block format, journal ordering, envelope validation, and pure scheduler policy are ordinary C/POSIX logic and should transfer subject to filesystem semantics. The cryptographic provider layer depends on OpenSSL and liboqs APIs. The accelerator layer depends on CUDA and cuPQC. The telemetry and direct-I/O layer is currently absent. Treating these four layers as equally portable would hide the core risk: the final two are exactly where drivers, hardware topology, and version specific behavior matter most.

Replication should thus begin with the CPU-only format on a second platform, then validate accelerator parity, then remeasure primitive placement, and only then connect platform-specific telemetry. No value in this paper should be used as a substitute for that replication.

A.13 Threat-to-mitigation matrix

The following matrix makes the intended security boundary operational. It also identifies why the prototype cannot inherit stronger claims from a TPM, CUDA, or FUSE component merely by linking against it. A mount credential is assumed secret for the lifetime of the process. The backing store is assumed untrusted. The operating system, CPU cryptographic library, CUDA runtime, and GPU driver are trusted computing-base components; their compromise is out of scope.

The matrix distinguishes authenticity from freshness. An authenticated file witness establishes only that someone with the mount key produced a record; it cannot establish newest state when an adversary restores every file-backed bit. Freshness needs protected external state, such as a provisioned TPM counter or a remote quorum. Conversely, that state alone does not address process compromise, credential release, or GPU side channels. The paper therefore

Table 6: Source-level audit of central mechanism claims.

Questionable mechanism	What the final source does	Permitted paper claim
CUDA access "permission flip"	`cudaMallocManaged`, explicit prefetch, and stream synchronization inside CUDA executors	Executor-local locality and ordering; no global CPU/GPU/DMA mutual-exclusion claim
NVMe `O_DIRECT` into UVM	FUSE sidecars use POSIX `pread`/`pwrite`; auxiliary `io_uring_uvm` validates same-buffer GPU visibility for pinned host memory	Auxiliary evidence only; no final FUSE data-path DMA or migration-suppression claim
`cudaHostRegister`/GUP pinning	Used in auxiliary probes and retained logs; the final storage path does not depend on it	Host-pinning and mapped-buffer checksum only
eBPF `nvme_complete_rq` trigger	No eBPF program, kprobe, or kernel notification path in the final implementation	Not implemented or evaluated
io_uring completion path	No io_uring submission/completion handling in FUSE storage path	Not implemented or evaluated
ML-KEM per-block encryption	AES-GCM protects blocks; ML-KEM executors are optional batch-oriented code	ML-KEM primitive benchmark only
TPM "signing"	Optional NV record backend; no signing primitive is claimed	Pre-provisioning requirement and unmeasured experimental path
TVLA/constant time	Synthetic trace generators excluded from paper	No side-channel-resistance claim

Table 7: Threat boundary and current response.

Event or adversary action	Current response	Residual risk
Read backing data sidecar	AES-GCM ciphertext; DEK envelope remains wrapped	Weak/reused mount password and offline password guessing remain deployment concerns
Modify ciphertext or journal mapping	GCM tag or HMAC checkpoint/envelope validation fails closed	Denial of service; incomplete fault coverage
Restore all file-backed state	Baseline snapshot remains readable; classified as rollback visible	No freshness protection without external state
Request unconfigured TPM backend	Initialization fails before mounting	No tested provisioning, authorization, or recovery workflow
Omit AI telemetry/slack budget	Scheduler selects CPU	No opportunistic GPU scheduling or measured QoS benefit
Co-resident GPU observer	No mitigation claimed	Timing, cache, occupancy, power, and EM channels out of scope
Compromise FUSE process or CUDA driver	Out of scope	Can access plaintext/keys or alter execution

never presents local integrity as a rollback or constant-time guarantee.

References

[1] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W. Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 387–401.

[2] Clemens Fruhwirth. 2005. New methods in hard disk encryption: dm-crypt and LUKS. *Linux Journal* 2005, 138 (2005).

[3] Michael Halcrow, Theodore Ts'o, et al. 2015. Ext4 Encryption: Design and Implementation of Linux Filesystem Encryption. *Proceedings of the Linux Symposium* (2015).

[4] Borui Li, Tiange Xia, and Shuai Wang. 2025. InfScaler: Enabling Efficient ML Inference Serving on Multi-Accelerator Edge Devices. In *ACM/IEEE Design Automation Conference (DAC)*.

[5] National Institute of Standards and Technology. 2015. *FIPS 180-4: Secure Hash Standard (SHS)*. Technical Report. NIST.

[6] National Institute of Standards and Technology. 2024. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Technical Report. NIST.

[7] NVIDIA Corporation. 2024. *NVIDIA Jetson Orin Nano Developer Kit Platform Specifications*. https://developer.nvidia.com/embedded/jetson-orin-nano-developer-kit

[8] OP-TEE. 2018. OP-TEE: Open Portable Trusted Execution Environment Specification. In *GlobalPlatform Technology Conference*.

[9] Daeyoung Park, Heehoon Kim, Jinpyo Kim, Taehyun Kim, and Jaejin Lee. 2022. SnuQS: Out-of-Core Cache Staging for Virtualized Memory Storage Systems. In *Proceedings of the International Conference on Supercomputing*.

[10] Jakob Spenneberg et al. 2017. gocryptfs: A Modern FUSE-based Encrypted Filesystem. In *USENIX Security Association*.

[11] Jia Wei, Xingjun Zhang, Longxiang Wang, and Zheng Wei. 2023. Fastensor: Optimize the Tensor I/O Path from SSD to GPU for Deep Learning Training. *ACM Trans. Archit. Code Optim.* 20, 4 (2023).

[12] Feng Zhang, Chenyang Zhang, Jiawei Guan, Qiangjun Zhou, Kuangyu Chen, Xiao Zhang, Bingsheng He, Jidong Zhai, and Xiaoyong Du. 2025. Breaking the Edge: Enabling Efficient Neural Network Inference on Integrated Edge Devices. *IEEE Transactions on Cloud Computing* (2025).

[13] Xingjun Zhang, Longxiang Wang, et al. 2023. fscrypt-GPU: GPU-Accelerated Filesystem Encryption for High-Throughput Secure Storage. In *ACM Symposium on Cloud Computing*.